

Actividad: Introducción a la POO en PHP

Autor: Ismael Ponce Gordillo 17-Nov-2025 Licencia Creative Commons v4.0 BY-SA-NC

Sumario

Actividad: Introducción a la POO en PHP.....	1
Contexto de la Actividad.....	1
PARTE 1: Fundamentos - Tu Primera Clase.....	1
Paso 1: Crear la Estructura Básica.....	1
Paso 2: Crear la Clase Libro - Libro.php.....	1
Conceptos Clave:.....	2
PARTE 2: Usar la Clase.....	2
Paso 3: Crear el Archivo Principal - index.php.....	2
Conceptos Clave:.....	3
PARTE 3: Propiedades Privadas y Encapsulación.....	4
Paso 4: Mejorar la Clase con Encapsulación.....	4
Paso 5: Usar la Clase Mejorada - ejemplo_encapsulacion.php.....	5
Conceptos Clave:.....	6
PARTE 4: Ejercicios Prácticos.....	7
Ejercicio 1: Crear la Clase Socio.....	7
Ejercicio 2: Crear la Clase Prestamo.....	7
Ejercicio 3: Sistema Completo.....	7
PARTE 5: Comparación Propiedades Públicas vs Privadas.....	8
PARTE 6: Preguntas de Comprensión.....	8
Conceptos para Recordar.....	8

Contexto de la Actividad

Vas a desarrollar un **sistema simple de biblioteca** donde podrás crear libros, mostrar su información y realizar operaciones básicas como préstamos.

PARTE 1: Fundamentos - Tu Primera Clase

Paso 1: Crear la Estructura Básica

Crea una carpeta llamada biblioteca y dentro de ella crea un archivo llamado Libro.php:

```
biblioteca/  
├── Libro.php  
└── index.php
```

Paso 2: Crear la Clase Libro - Libro.php

```
<?php
```

```
/** Clase Libro Representa un libro de la biblioteca */
```

```
class Libro {
```

```
    // PROPIEDADES (atributos del libro)
```

```
    public $titulo;
```

```
    public $autor;
```

```
    public $numeroPaginas;
```

```
    public $disponible;
```

```
    /** CONSTRUCTOR Se ejecuta automáticamente al crear un nuevo objeto Libro */
```

```
    public function __construct($titulo, $autor, $numeroPaginas) {
```

```
        $this->titulo = $titulo;
```

```
        $this->autor = $autor;
```

```
        $this->numeroPaginas = $numeroPaginas;
```

```
        $this->disponible = true; // Por defecto, el libro está disponible
```

```
    }
```

```
    /** MÉTODO: Muestra la información del libro */
```

```

public function mostrarInfo() {
    $estado = $this->disponible ? "Disponible" : "Prestado";
    echo "<div style='border: 2px solid #333; padding: 15px; margin: 10px 0; border-radius: 5px;'>";
    echo "<h3>ⓧ {$this->titulo}</h3>";
    echo "<p><strong>Autor:</strong> {$this->autor}</p>";
    echo "<p><strong>Páginas:</strong> {$this->numeroPaginas}</p>";
    echo "<p><strong>Estado:</strong> {$estado}</p>";
    echo "</div>";
}
/** MÉTODO: Prestar el libro */
public function prestar() {
    if ($this->disponible) {
        $this->disponible = false;
        echo "<p style='color: green;'>✓ El libro '{$this->titulo}' ha sido prestado.</p>";
    } else {
        echo "<p style='color: red;'>✗ El libro '{$this->titulo}' ya está prestado.</p>";
    }
}
/** MÉTODO: Devolver el libro */
public function devolver() {
    if (!$this->disponible) {
        $this->disponible = true;
        echo "<p style='color: green;'>✓ El libro '{$this->titulo}' ha sido devuelto.</p>";
    } else {
        echo "<p style='color: orange;'>⚠ El libro '{$this->titulo}' ya estaba disponible.</p>";
    }
}
}
?>

```

Conceptos Clave:

- **Clase:** Es como un “molde” o “plantilla” para crear objetos
- **Propiedades:** Variables que pertenecen a la clase (características del libro)
- **public:** Significa que se puede acceder desde fuera de la clase
- **\$this:** Hace referencia al objeto actual
- **Constructor:** Método especial que se ejecuta al crear un objeto
- **Métodos:** Funciones que pertenecen a la clase (acciones que puede hacer el libro)

PARTE 2: Usar la Clase

Paso 3: Crear el Archivo Principal - index.php

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sistema de Biblioteca - POO</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 800px; margin: 50px auto; padding: 20px; background: #f4f4f4; }
        h1 { text-align: center; color: #333; }
        .seccion { background: white; padding: 20px; margin: 20px 0; border-radius: 10px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
        h2 { color: #007bff; border-bottom: 2px solid #007bff; padding-bottom: 10px; }
    </style>
</head>
<body>
    <h1>Sistema de Biblioteca - POO en PHP</h1>

    <?php

```

```
// IMPORTANTE: Incluir la clase Libro
require_once 'Libro.php';
echo "<div class='seccion'>";
echo "<h2>1. Crear Objetos (Instancias de la clase)</h2>"; // Crear el primer libro
$libro1 = new Libro("Don Quijote de la Mancha", "Miguel de Cervantes", 863); // Crear el segundo libro
$libro2 = new Libro("Cien años de soledad", "Gabriel García Márquez", 471); // Crear el tercer libro
$libro3 = new Libro("1984", "George Orwell", 326);
echo "<p>✓ Se han creado 3 libros en la biblioteca</p>";
echo "</div>";

// ===== MOSTRAR INFORMACIÓN DE LOS LIBROS
echo "<div class='seccion'>";
echo "<h2>2. Mostrar información usando métodos</h2>";
$libro1->mostrarInfo();
$libro2->mostrarInfo();
$libro3->mostrarInfo();
echo "</div>";

// ===== ACCESO DIRECTO A PROPIEDADES PÚBLICAS
echo "<div class='seccion'>";
echo "<h2>3. Acceso directo a propiedades públicas</h2>";
echo "<p>Título del libro 1: <strong>{$libro1->titulo}</strong></p>";
echo "<p>Autor del libro 2: <strong>{$libro2->autor}</strong></p>";
echo "<p>Páginas del libro 3: <strong>{$libro3->numeroPaginas}</strong></p>";
// También podemos MODIFICAR propiedades directamente
echo "<hr>";
echo "<p>✎ Cambiando el número de páginas del libro 1...</p>";
$libro1->numeroPaginas = 900;
echo "<p>Nuevo número de páginas: <strong>{$libro1->numeroPaginas}</strong></p>";
echo "</div>";

// ===== USAR MÉTODOS PARA CAMBIAR ESTADO
echo "<div class='seccion'>";
echo "<h2>4. Usar métodos para realizar acciones</h2>"; // Prestar libros
$libro1->prestar();
$libro2->prestar();
echo "<hr>"; // Intentar prestar un libro ya prestado
echo "<p><strong>Intentando prestar el libro 1 otra vez:</strong></p>";
$libro1->prestar();
echo "<hr>"; // Devolver libros
echo "<p><strong>Devolviendo libros:</strong></p>";
$libro1->devolver();
echo "</div>";

// ===== ESTADO FINAL DE LOS LIBROS
echo "<div class='seccion'>";
echo "<h2>5. Estado final de todos los libros</h2>";
$libro1->mostrarInfo();
$libro2->mostrarInfo();
$libro3->mostrarInfo();
echo "</div>";
?>

</body>
</html>
```

Conceptos Clave:

- new: Palabra clave para crear un nuevo objeto de una clase
- **Instancia:** Un objeto creado a partir de una clase
- ->: Operador para acceder a propiedades y métodos de un objeto
- require_once: Incluye un archivo PHP (la clase) una sola vez

PARTE 3: Propiedades Privadas y Encapsulación

Paso 4: Mejorar la Clase con Encapsulación

Ahora vamos a crear una versión mejorada de la clase Libro con propiedades **privadas** y métodos **getters/setters**.

Crea un nuevo archivo llamado LibroMejorado.php:

```
<?php
/** Clase LibroMejorado Versión mejorada con encapsulación */
class LibroMejorado { // PROPIEDADES PRIVADAS (solo accesibles desde dentro de la clase)
    private $titulo;
    private $autor;
    private $numeroPaginas;
    private $disponible;
    private $isbn;
    /** CONSTRUCTOR */
    public function __construct($titulo, $autor, $numeroPaginas, $isbn) {
        $this->titulo = $titulo;
        $this->autor = $autor;
        $this->numeroPaginas = $numeroPaginas;
        $this->isbn = $isbn;
        $this->disponible = true;
    }
    // ===== GETTERS (métodos para OBTENER valores)
    public function getTitulo() { return $this->titulo; }
    public function getAutor() { return $this->autor; }
    public function getNumeroPaginas() { return $this->numeroPaginas; }
    public function getIsbn() { return $this->isbn; }
    public function isDisponible() { return $this->disponible; }
    // ===== SETTERS (métodos para MODIFICAR valores)
    public function setTitulo($titulo) {
        if (strlen($titulo) > 3) {
            $this->titulo = $titulo;
            return true;
        } else {
            echo "<p style='color: red;'>✘ El título debe tener al menos 3 caracteres</p>";
            return false;
        }
    }
    public function setNumeroPaginas($numero) {
        if ($numero > 0) {
            $this->numeroPaginas = $numero;
            return true;
        } else {
            echo "<p style='color: red;'>✘ El número de páginas debe ser positivo</p>";
            return false;
        }
    }
    // ===== OTROS MÉTODOS
    public function mostrarInfo() {
        $estado = $this->disponible ? "Disponible" : "Prestado";
        echo "<div style='border: 2px solid #28a745; padding: 15px; margin: 10px 0; border-radius: 5px; background: #f8f9fa;'>";
        echo "<h3>📖 {$this->titulo}</h3>";
        echo "<p><strong>Autor:</strong> {$this->autor}</p>";
        echo "<p><strong>ISBN:</strong> {$this->isbn}</p>";
        echo "<p><strong>Páginas:</strong> {$this->numeroPaginas}</p>";
        echo "<p><strong>Estado:</strong> <span style='color: " . ($this->disponible ? 'green' : 'red') . ">{$estado}</span></p>";
        echo "</div>";
    }
}
```

```

public function prestar() {
    if ($this->disponible) {
        $this->disponible = false;
        return true;
    }
    return false;
}

public function devolver() {
    if (!$this->disponible) {
        $this->disponible = true;
        return true;
    }
    return false;
}

/** Método para calcular tiempo de lectura estimado (basado en 1 página por minuto) */
public function tiempoLecturaEstimado() {
    $horas = floor($this->numeroPaginas / 60);
    $minutos = $this->numeroPaginas % 60;
    return "{$horas} horas y {$minutos} minutos";
}
}
?>

```

Paso 5: Usar la Clase Mejorada - ejemplo_encapsulacion.php

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Encapsulación en POO</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 900px; margin: 50px auto; padding: 20px; background: #f4f4f4; }
        h1 { text-align: center; color: #333; }
        .seccion { background: white; padding: 20px; margin: 20px 0; border-radius: 10px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
        h2 { color: #28a745; border-bottom: 2px solid #28a745; padding-bottom: 10px; }
        .codigo { background: #f5f5f5; padding: 10px; border-left: 4px solid #28a745; font-family: monospace; margin: 10px 0; }
    </style>
</head>
<body>
    <h1>Encapsulación y Getters/Setters</h1>
    <?php
        require_once 'LibroMejorado.php';
        echo "<div class='seccion'>";
        echo "<h2>1. Crear libro con propiedades privadas</h2>";
        $libro = new LibroMejorado("El Principito", "Antoine de Saint-Exupéry", 96, "978-0156012195" );
        echo "<p>✔ Libro creado correctamente</p>";
        $libro->mostrarInfo();
        echo "</div>";

        // ===== INTENTAR ACCESO DIRECTO (NO FUNCIONARÁ)
        echo "<div class='seccion'>";
        echo "<h2>2. Diferencia entre propiedades públicas y privadas</h2>";
        echo "<p><strong>✗ Si intentamos acceder directamente a una propiedad privada:</strong></p>";
        echo "<div class='codigo'>";
        echo "// echo \$libro->titulo; <-- Esto daría ERROR<br>";
        echo "// PHP Fatal error: Cannot access private property";
        echo "</div>";
        echo "<p><strong>✔ Debemos usar los métodos GETTER:</strong></p>";
        echo "<div class='codigo'>";
        echo "echo \$libro->getTitulo(); <-- Esto funciona correctamente";
        echo "</div>";
        echo "<p>Título del libro: <strong>" . $libro->getTitulo() . "</strong></p>";
    </?php>

```

```

echo "<p>Autor del libro: <strong>" . $libro->getAutor() . "</strong></p>";
echo "<p>ISBN: <strong>" . $libro->getIsbn() . "</strong></p>";
echo "</div>";
// ===== USAR SETTERS CON VALIDACIÓN
echo "<div class='seccion'>";
echo "<h2>3. Modificar propiedades usando SETTERS (con validación)</h2>";
echo "<p><strong>Intentando cambiar el título a algo válido:</strong></p>";
$libro->setTitulo("El Principito - Edición Ilustrada");
echo "<p>Nuevo título: <strong>" . $libro->getTitulo() . "</strong></p>";
echo "<hr>";
echo "<p><strong>Intentando cambiar el título a algo inválido (menos de 3 caracteres):</strong></p>";
$libro->setTitulo("El");
echo "<hr>";
echo "<p><strong>Intentando cambiar el número de páginas a un valor negativo:</strong></p>";
$libro->setNumeroPaginas(-50);
echo "<hr>";
echo "<p><strong>Cambiano el número de páginas a un valor válido:</strong></p>";
$libro->setNumeroPaginas(120);
echo "<p>Nuevo número de páginas: <strong>" . $libro->getNumeroPaginas() . "</strong></p>";
echo "</div>";
// ===== USAR MÉTODOS ADICIONALES
echo "<div class='seccion'>";
echo "<h2>4. Métodos adicionales de la clase</h2>";
echo "<p>⌚ Tiempo de lectura estimado: <strong>" . $libro->tiempoLecturaEstimado() . "</strong></p>";
echo "</div>";
// ===== ESTADO FINAL
echo "<div class='seccion'>";
echo "<h2>5. Estado final del libro</h2>";
$libro->mostrarInfo();
echo "</div>";
?>
<div class="seccion">
  <h2>¿Por qué usar PRIVATE y getters/setters?</h2>
  <ul>
    <li><strong>Control:</strong> Podemos validar los datos antes de modificarlos</li>
    <li><strong>Seguridad:</strong> Evitamos que se modifiquen datos de forma incorrecta</li>
    <li><strong>Flexibilidad:</strong> Podemos cambiar cómo funcionan internamente sin afectar el código externo</li>
    <li><strong>Mantenimiento:</strong> Es más fácil encontrar errores y hacer cambios</li>
  </ul>
</div>
</body>
</html>

```

Conceptos Clave:

- **private:** Solo se puede acceder desde dentro de la clase
- **Encapsulación:** Ocultar los detalles internos de la clase
- **Getters:** Métodos para obtener valores de propiedades privadas
- **Setters:** Métodos para modificar valores con validación
- **Validación:** Comprobar que los datos sean correctos antes de guardarlos

PARTE 4: Ejercicios Prácticos

Ejercicio 1: Crear la Clase Socio

Crea una clase Socio que represente a un miembro de la biblioteca con:

Propiedades privadas: - nombre - dni - email - fechaRegistro

Métodos: - Constructor que inicialice todas las propiedades - Getters para todas las propiedades - Setter para email (con validación de que contenga "@") - mostrarInfo() que muestre los datos del socio

Archivo: Socio.php

```
<?php
class Socio {
    // TODO: Añade las propiedades privadas
    // TODO: Añade el constructor
    // TODO: Añade los getters
    // TODO: Añade el setter para email con validación
    // TODO: Añade el método mostrarInfo()
}
```

Ejercicio 2: Crear la Clase Prestamo

Crea una clase Prestamo que gestione el préstamo de un libro a un socio:

Propiedades públicas: - libro (objeto de tipo LibroMejorado) - socio (objeto de tipo Socio) - fechaPrestamo - fechaDevolucion

Métodos: - Constructor - realizarPrestamo(): marca el libro como prestado - finalizarPrestamo(): marca el libro como disponible - mostrarInfo(): muestra la información del préstamo

Ejercicio 3: Sistema Completo

Crea archivo sistema_completo.php que: 1. Cree 3 socios 2. Cree 3 libros 3. Realice 2 préstamos 4. Muestre toda la información

PARTE 5: Comparación Propiedades Públicas vs Privadas

```
<?php
// ===== CLASE CON PROPIEDADES PÚBLICAS (✗ MALA PRÁCTICA)
class LibroPublico {
    public $precio;
    public function __construct($precio) { $this->precio = $precio; }
}
$libro1 = new LibroPublico(20.50);
echo "Precio inicial: " . $libro1->precio . "€<br>"; // ¡PROBLEMA! Cualquiera puede poner un precio negativo
$libro1->precio = -100;
echo "Precio modificado: " . $libro1->precio . "€<br>"; // -100€ ✗
// ===== CLASE CON PROPIEDADES PRIVADAS (✓ BUENA PRÁCTICA)
class LibroPrivado {
    private $precio;
    public function __construct($precio) { $this->setPrecio($precio); }
    public function getPrecio() { return $this->precio; }
    public function setPrecio($precio) {
        if ($precio >= 0) {
            $this->precio = $precio;
        } else { echo "Error: El precio no puede ser negativo<br>"; }
    }
}
$libro2 = new LibroPrivado(20.50);
echo "Precio inicial: " . $libro2->getPrecio() . "€<br>";
// ✓ La validación impide precios negativos
$libro2->setPrecio(-100); // Muestra error
echo "Precio después de intento: " . $libro2->getPrecio() . "€<br>"; // 20.50€
?>
```

PARTE 6: Preguntas de Comprensión

Responde estas preguntas:

1. ¿Qué diferencia hay entre una clase y un objeto (instancia)?
2. ¿Para qué sirve el constructor de una clase?
3. ¿Cuál es la diferencia entre una propiedad pública y una privada?
4. ¿Por qué es mejor usar getters y setters en lugar de acceder directamente a las propiedades?
5. ¿Qué significa la palabra clave \$this en PHP?
6. Si tienes una clase Coche con una propiedad privada \$velocidad, ¿cómo obtendrías su valor desde fuera de la clase?

Conceptos para Recordar

Concepto	Descripción
Clase	Plantilla o molde para crear objetos
Objeto	Instancia de una clase
Propiedad	Variable que pertenece a una clase
Método	Función que pertenece a una clase
Constructor	Método especial que se ejecuta al crear un objeto
\$this	Referencia al objeto actual
public	Accesible desde cualquier lugar
private	Solo accesible desde dentro de la clase
Getter	Método para obtener el valor de una propiedad privada
Setter	Método para modificar el valor de una propiedad privada
Encapsulación	Ocultar detalles internos y exponer solo lo necesario